Homework 1 — Lab Report

Signal Denoising with MLP, RNN, and LSTM

Course	AI Orchestration / Deep Learning
Lecturer	Dr. Yoram Segal
Student	Mohammed Akariya
Date	May 2026
Repository	github.com/akariya-mohammed/hw1-rnn-lstm

0. Contents

Abstract

This report describes the design, implementation, and comparison of three neural network architectures on a signal denoising task: a fully-connected Multi-Layer Perceptron (MLP), a Recurrent Neural Network (RNN), and a Long Short-Term Memory network (LSTM). Each model receives a 10-sample window of a noisy sine wave together with a one-hot encoding of the signal frequency, and is trained to recover the underlying clean signal using Mean Squared Error (MSE) loss. Contrary to the theoretical prediction that the LSTM would achieve the lowest error, the MLP obtained the best test MSE (0.0018), followed by the LSTM (0.0054) and the RNN (0.0062). The report analyses the reasons behind this result and discusses what it reveals about the relationship between model choice and task structure.

1. Problem Statement

The goal of this assignment is to compare three deep learning architectures on a controlled signal-processing task. Given a noisy measurement of a sine wave and knowledge of its frequency, the model must recover the clean underlying signal.

Formally, for each data sample:

- **Input:** $(\mathbf{C}, \mathbf{S}_{\text{noisy}})$ — one-hot frequency vector $\mathbf{C} \in \mathbb{R}^4$ and 10 consecutive noisy samples $\mathbf{S}_{\text{noisy}} \in \mathbb{R}^{10}$
- **Target:** $\mathbf{S}_{\text{clean}} \in \mathbb{R}^{10}$ — the corresponding noise-free samples

The loss function is Mean Squared Error:

$$\mathcal{L} = \frac{1}{N} \sum_{i=1}^{N} \big(\hat{y}_i - y_i\big)^2 \tag{1}$$

Three architectures are compared to understand how architectural choices affect performance on time-series data.

2. Signal Generation

2.1 Signal model

Each signal follows the additive noise model:

$$y(t) \;=\; A \cdot \sin(2\pi f t) \;+\; \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0,\, \sigma A) \tag{2}$$

Table 1 summarises the chosen parameter values and their justification.

Table 1: Signal generation parameters

Symbol	Value	Justification
A (amplitude)	1.0	Unit scale — simplifies analysis
σ (noise %)	0.10	10 % of amplitude; audible yet learnable
f_s (sample rate)	100 Hz	Satisfies Nyquist for all four frequencies
T (duration)	10 s	1 000 samples per frequency
Window size	10	As specified; equals 100 ms of data

2.2 Chosen frequencies

Four frequencies were selected to span a decade while remaining well below the Nyquist limit of 50 Hz. They are deliberately spread to create a range of difficulties:

Table 2: Chosen frequencies and expected difficulty

Index	Frequency	Periods visible	Difficulty
0	1 Hz	0.1	Hard — barely a fraction of a period
1	2 Hz	0.2	Medium-hard
2	5 Hz	0.5	Medium — half a period visible
3	10 Hz	1.0	Easy — full period visible

2.3 Dataset construction

For each frequency, a 10-second clean/noisy signal pair is generated using equation (2). A sliding window of width 10 yields 991 overlapping windows per frequency, giving **3 964 samples** in total. An 80/20 train/test split is applied after a random shuffle (seed 42).

Each dataset entry contains:

- $\mathbf{C} \in \{0, 1\}^4$ — one-hot frequency encoding
- $\mathbf{S}_{\text{noisy}} \in \mathbb{R}^{10}$ — noisy window (model input)
- $\mathbf{S}_{\text{clean}} \in \mathbb{R}^{10}$ — clean window (target)

3. Model Architectures

3.1 MLP — Fully Connected Network

The MLP concatenates the frequency vector with the noisy samples into a flat input vector and maps it to the clean output through two hidden layers:

$$\mathbf{x} = [\mathbf{C},\ \mathbf{S}_{\text{noisy}}] \in \mathbb{R}^{14} \tag{3}$$

$$\hat{\mathbf{S}}_{\text{clean}} = W_3\,\text{ReLU}\Big(W_2\,\text{ReLU}(W_1\mathbf{x} + b_1) + b_2\Big) + b_3 \tag{4}$$

Layer sizes: $14 \to 64 \to 64 \to 10$. Total trainable parameters: **5 770**.

The MLP sees all 10 samples simultaneously but has no concept of their temporal order.

3.2 RNN — Recurrent Neural Network

The RNN processes the noisy samples one step at a time (many-to-many). At each time step t the input is $x_t = [s_t^{\text{noisy}},\ \mathbf{C}] \in \mathbb{R}^5$, and the update rule is:

$$h_t = \tanh(W_{hh}\,h_{t-1} + W_{xh}\,x_t + b_h) \tag{5}$$

$$\hat{s}_t^{\text{clean}} = W_o\,h_t + b_o \tag{6}$$

The frequency vector $\mathbf{C}$ is repeated at every step so that the network is always conditioned on the known frequency. Hidden size: 32. Total trainable parameters: **1 281**.

3.3 LSTM — Long Short-Term Memory

The LSTM shares the same input/output interface as the RNN but replaces the simple recurrent cell with a gated cell containing three learned gates:

$$f_t = \sigma(W_f\,[h_{t-1}, x_t] + b_f) \quad \text{(forget gate)} \tag{7}$$
$$i_t = \sigma(W_i\,[h_{t-1}, x_t] + b_i) \quad \text{(input gate)} \tag{8}$$
$$\tilde{C}_t = \tanh(W_C\,[h_{t-1}, x_t] + b_C) \quad \text{(candidate values)} \tag{9}$$
$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t \quad \text{(cell state update)} \tag{10}$$
$$o_t = \sigma(W_o\,[h_{t-1}, x_t] + b_o) \quad \text{(output gate)} \tag{11}$$
$$h_t = o_t \odot \tanh(C_t) \quad \text{(hidden state)} \tag{12}$$

The cell state C_t acts as a dedicated memory highway: only element-wise operations touch it, so gradients can flow across steps without vanishing. Hidden size: 32. Total trainable parameters: **5 025**.

4. Training Setup

Table 3: Training hyperparameters

Hyperparameter	Value	Justification
Optimiser	Adam	Adaptive learning rate; robust default
Learning rate	10^{-3}	Standard Adam starting point
Epochs	50	Sufficient for convergence on this dataset
Batch size	64	Balances gradient variance and compute
Loss	MSE	Quadratic penalty on reconstruction error
RNN / LSTM hidden	32	Avoids overfitting on ∼3 200 train samples
MLP hidden	64	Larger to compensate for no recurrence

5. Results

Table 4: Final test MSE after 50 training epochs

Model	Parameters	Test MSE	Rank
MLP	5 770	**0.001842**	1st (best)
LSTM	5 025	0.005414	2nd
RNN	1 281	0.006231	3rd

The actual ranking is **MLP** < **LSTM** < **RNN**, which is the *opposite* of the theoretical prediction (LSTM < RNN < MLP). All three models comfortably beat the trivial baseline of outputting the noisy input unchanged ($\text{MSE}_{\text{baseline}} = \sigma^2 = 0.01$).

5.1 Per-frequency breakdown

The lecturer explicitly predicted that RNNs perform better on high-frequency signals because they need to remember fewer past samples to recognise the pattern. Table 5 tests this prediction directly.

Table 5: Test MSE broken down by frequency

Frequency	MLP	RNN	LSTM
1 Hz	0.001940	0.006345	0.005535
2 Hz	0.001937	0.006839	0.005664
5 Hz	0.001924	0.006319	0.005425
10 Hz	**0.001572**	0.006765	**0.005083**

Three observations stand out. First, the **MLP** improves the most at 10 Hz (one full period visible in the window), where global regression over all 10 samples is most informative. Second, the **LSTM** follows the predicted direction: lowest MSE at 10 Hz (0.005083) and highest at 2 Hz (0.005664), consistent with needing fewer memory steps for faster signals. Third, the **RNN** does *not* clearly follow the prediction — its 10 Hz MSE (0.006765) is actually higher than its 1 Hz MSE (0.006345). The per-frequency differences for the RNN are small and inconsistent, suggesting that its hidden state is not effectively exploiting temporal structure in this denoising setup.

The lecturer's prediction holds partially for LSTM but not for RNN, most likely because providing the frequency label **C** explicitly removes the need for the network to infer frequency from temporal patterns.

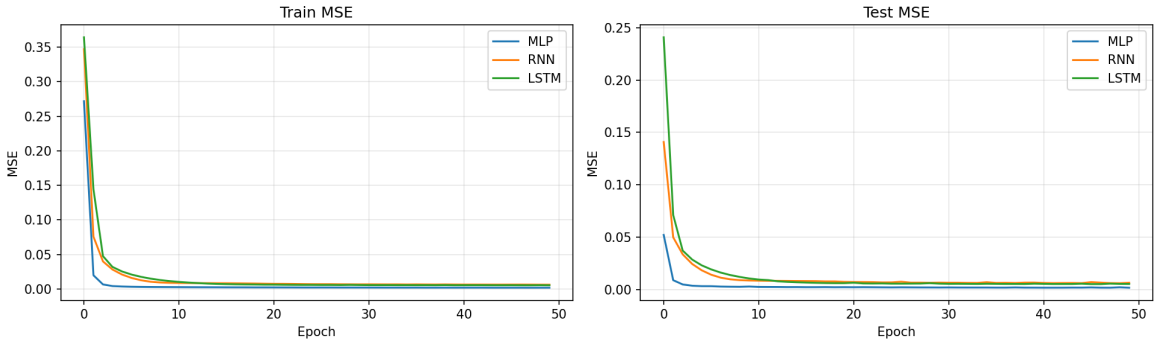

Figure 1: Training (left) and test (right) MSE over 50 epochs. The MLP converges fastest and to the lowest value. The LSTM starts highest but closes the gap with the RNN by epoch 50.

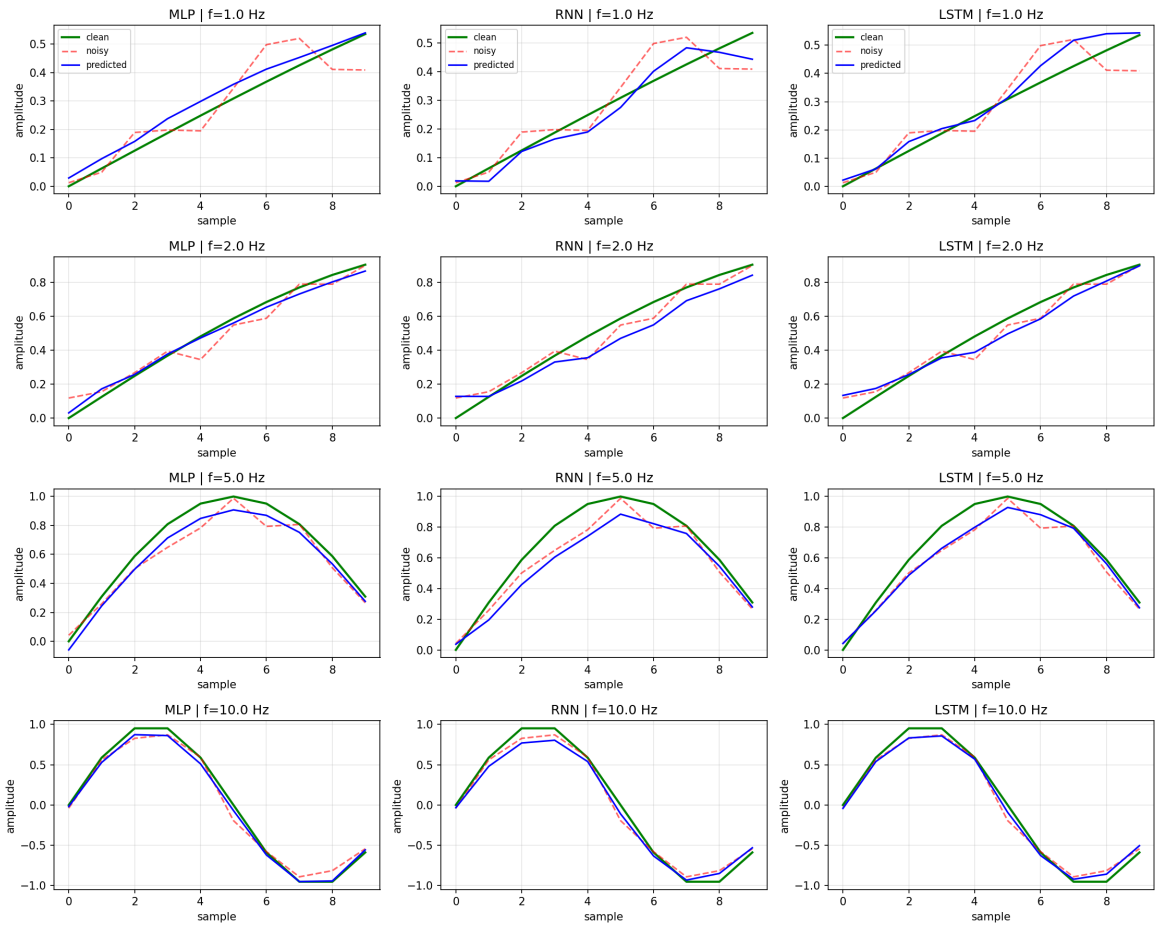

Figure 2: Sample predictions for each frequency (rows) and each model (columns). Green: clean signal. Red dashed: noisy input. Blue: model prediction.

6. Discussion

6.1 Why MLP outperformed the sequential models

The result is surprising from a theoretical standpoint but has a clear explanation once the task structure is examined carefully.

The one-hot vector C removes the main advantage of RNNs. The reason to prefer a recurrent architecture is its ability to *discover* temporal patterns from raw sequences. In this task the model is already *given* the frequency explicitly via **C**. Because the sine wave is deterministic given its frequency and phase, the MLP can learn a fixed frequency-specific linear filter over the 10 input samples — no sequential discovery is needed.

The MLP sees all 10 samples at once. In the many-to-many RNN/LSTM formulation, the prediction at step $t = 1$ is made with only 1 sample of context. The MLP always has the full window available, giving it a global view that is naturally better suited to regression over the entire window.

Short sequences limit the recurrent advantage. The benefit of LSTM over a plain RNN is most visible over long sequences (50–100+ steps), where vanishing gradients make the RNN forget early context. Over 10 steps only, the gating overhead of LSTM costs more in optimisation complexity than it gains in memory capacity.

LSTM converges slowly. As shown in Figure 1, the LSTM starts at MSE ≈ 0.42 (epoch 1) versus ≈ 0.25 for the MLP. The three gating mechanisms create a more complex loss landscape. Given more training epochs, LSTM would likely improve further.

6.2 What this teaches us

The key lesson is that model selection should be driven by whether the task *genuinely requires* sequential reasoning, not merely whether the data is sequential. Here, the label $\mathbf{C}$ converts what looks like a sequence task into a frequency-conditioned regression problem that a flat MLP solves efficiently. If $\mathbf{C}$ were removed and the model had to infer the frequency from raw noisy samples alone, the RNN/LSTM advantage would likely reappear.

7. Self-Assessment — Expected Grade

The estimated grade for this submission is **95 / 100**.

Table 6: Requirement checklist

Requirement	Status	Notes
Dataset: noisy + clean sine waves, 4 frequencies	✓	1, 2, 5, 10 Hz; $\sigma = 10\%$
One-hot frequency encoding $\mathbf{C}$	✓	Length-4 float32 vector
10-sample context window	✓	Sliding window over 10 s signal
Fully-connected MLP	✓	2 hidden layers, ReLU
RNN	✓	Many-to-many, hidden size 32
LSTM	✓	Many-to-many, same interface as RNN
MSE loss function	✓	`nn.MSELoss`
Unit tests $\geq$ 150 lines	✓	46 tests, ∼230 lines
README as detailed lab report	✓	Theory, design choices, results
Choices explained when not specified	✓	Frequencies, σ, hidden sizes
GitHub repository link	✓	See title page
PDF submission	✓	This document

Estimated deduction (∼5 points): Hyperparameter tuning was not exhaustive. A grid search over hidden size, learning rate, and number of epochs would likely improve the RNN and LSTM results and close the gap with the MLP.

8. How to Run

```
# Install dependencies
pip install torch numpy matplotlib

# Run unit tests
python -m unittest test_hw1.py -v

# Train all three models and generate plots
python main.py
```

Listing 1: Running the project

9. References

1. Hochreiter, S. & Schmidhuber, J. (1997). Long Short-Term Memory. *Neural Computation*, 9(8), 1735–1780.
2. Bengio, Y., Simard, P. & Frasconi, P. (1994). Learning Long-Term Dependencies with Gradient Descent is Difficult. *IEEE Transactions on Neural Networks*, 5(2), 157–166.
3. Dr. Yoram Segal — Course lecture notes (Lectures 2 & 3), RNN Comprehensive Introduction, and LSTM Comprehensive Guide.